

Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 7b - Model-Free RL (Part 2)

Sarah Keren

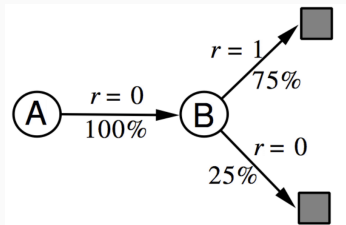
The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Temporal-Difference Learning

AB Example

Two states A, B; no discounting; 8 episodes of experience

- A, 0, B, 0
- B, 1
- B, 1
- B, 1
- B, 1
- B, 1
- B, 1
- B, 0



What is $V(A)$, $V(B)$?

From Monte Carlo to Temporal-Difference (TD) Learning

Temporal-difference (TD) learning has several advantages over Monte-Carlo (MC)

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

From Monte Carlo to Temporal-Difference (TD) Learning

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

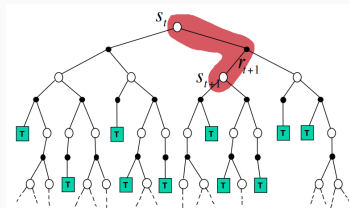
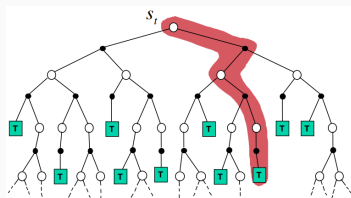
Temporal-difference (TD) learning has several advantages over Monte-Carlo (MC)

- Lower variance
- Online
- Works with incomplete sequences

Natural idea: use TD instead of MC in our control loop

- Apply TD to $Q(s, a)$
- Use ϵ -greedy policy improvement
- Update every time-step.

From Monte Carlo to Temporal-Difference (TD) Learning



Temporal-Difference Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Reminder: Bellman Formulas

Bellman equation (for a given deterministic policy):

Multiple variations, e.g.,

$$V_{\pi}(s) = R(s) + \gamma \sum_{s'} \mathcal{P}(s'|s, \pi(s)) \cdot V_{\pi}(s')$$

Bellman equation (for a given stochastic policy):

$$V_{\pi}(s) = \sum_a \pi(a | s) \left[R(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) V_{\pi}(s') \right]$$

Using Q

$$Q_{\pi}(s, a) = R(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) \sum_{a'} \pi(a' | s') Q_{\pi}(s', a')$$

Can we use these in RL?

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Reminder: Bellman Optimality Equations

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) V^*(s') \right]$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) \max_{a'} Q^*(s', a')$$

$$V^*(s) = \max_a Q^*(s, a)$$

Can we use these in RL?

Reminder: during the learning process, we see (possibly partial) trajectories of the form:

$$\tau = s_0, a_1, r_1, s_1, \dots, s_{T-1}, a_{T-1}, r_T, s_T$$

TD Evaluation

Temporal-Difference (TD) Evaluation

- For each transition from s to s' , we perform update

$$V_{\pi}(s) := V_{\pi}(s) + \alpha(R(s) + \gamma V_{\pi}(s') - V_{\pi}(s))$$

with α as the learning rate

How does this move us closer to satisfying the Bellman constraint ?

$$V_{\pi}(s) = R(s) + \gamma \sum_{s'} \mathcal{P}(s'|s, \pi(s)) \cdot V_{\pi}(s')$$

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Temporal-Difference (TD) Evaluation

- For each transition from s to s' , we perform update

$$V_{\pi}(s) := V_{\pi}(s) + \alpha(R(s) + \gamma V_{\pi}(s') - V_{\pi}(s))$$

with α as the learning rate

How does this move us closer to satisfying the Bellman constraint ?

$$V_{\pi}(s) = R(s) + \gamma \sum_{s'} \mathcal{P}(s'|s, \pi(s)) \cdot V_{\pi}(s')$$

- $R(s) + \gamma V_{\pi}(s')$ is a (noisy) sample of utility based on the next state.
- The update maintains a “mean” of (noisy) utility samples
- How do we guarantee convergence to the true values ?

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Temporal-Difference (TD) Evaluation

- For each transition from s to s' , we perform update

$$V_{\pi}(s) := V_{\pi}(s) + \alpha(R(s) + \gamma V_{\pi}(s') - V_{\pi}(s))$$

with α as the learning rate

How does this move us closer to satisfying the Bellman constraint ?

$$V_{\pi}(s) = R(s) + \gamma \sum_{s'} \mathcal{P}(s'|s, \pi(s)) \cdot V_{\pi}(s')$$

- $R(s) + \gamma V_{\pi}(s')$ is a (noisy) sample of utility based on the next state.
- The update maintains a “mean” of (noisy) utility samples
- How do we guarantee convergence to the true values ?
 - If the learning rate decreases appropriately with the number of samples (e.g. $\frac{1}{n}$), then the utility estimates will converge to true values (non-trivial).

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

TD Control

Temporal-Difference (TD) Control

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

- Simplest temporal-difference learning algorithm: $TD(0)$
 - Update value $V(s_t)$ toward estimated return $R_{t+1} + \gamma V(s_{t+1})$

$$V(s_t) \leftarrow V(s_t) + \alpha(R_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$

- If we are using Q

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

Which policy do we use here?

Temporal-Difference (TD) Control

Do local updates of utility/value function on a per-action basis.

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Temporal-Difference (TD) Control

Do local updates of utility/value function on a per-action basis.

What does TD learn from?

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Temporal-Difference (TD) Control

Do local updates of utility/value function on a per-action basis.

What does TD learn from? TD methods learn directly from episodes of experience

Model-free or model-based?

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Temporal-Difference (TD) Control

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Do local updates of utility/value function on a per-action basis.

What does TD learn from? TD methods learn directly from episodes of experience

Model-free or model-based? TD is model-free: no learning of MDP transitions / rewards (doesn't try to estimate entire transition function).

Is there a need to wait for the episode to end?

Temporal-Difference (TD) Control

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Do local updates of utility/value function on a per-action basis.

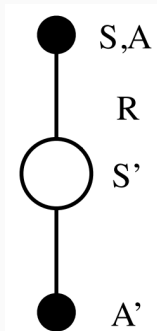
What does TD learn from? TD methods learn directly from episodes of experience

Model-free or model-based? TD is model-free: no learning of MDP transitions / rewards (doesn't try to estimate entire transition function).

Is there a need to wait for the episode to end? TD learns from incomplete episodes, by **bootstrapping** - updates a guess towards a guess.

On-Policy Control With SARSA

Updating action-value functions with



$$Q(S, A) \leftarrow Q(s, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$$

Temporal-
Difference
Learning

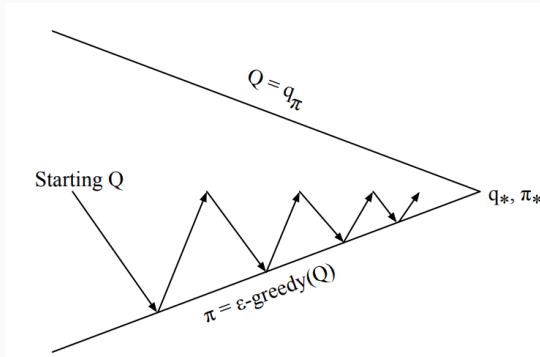
TD Evaluation

TD Control

MC vs. TD

Conclusion

On-Policy Control With SARSA



Every time-step:

- Policy evaluation: Sarsa $Q \approx Q_\pi$
- Policy improvement: α -greedy improvement

Temporal-Difference Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Algorithm 1 SARSA (On-Policy TD Control)

```
1: Initialize:
2:  $Q(s, a) \leftarrow$  arbitrary for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$ ,  $Q(\text{terminal}, \cdot) \leftarrow 0$ 
3: while true do
4:   Initialize start state  $S$ 
5:   Choose action  $A$  using an  $\epsilon$ -greedy policy derived from  $Q$ 
6:   while  $S$  is not terminal do
7:     Execute action  $A$ , observe reward  $R$  and next state  $S'$ 
8:     Choose action  $A'$  derived from  $Q$  (e.g.,  $\epsilon$ -greedy policy)
9:
10:     $Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$ 
11:     $S \leftarrow S'$ 
12:     $A \leftarrow A'$ 
12:   end while
13: end while
```

$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$$

Evaluate **target policy** $\pi(a|s)$ to compute $V_\pi(s)$ or $Q_\pi(s, a)$ while following **behaviour policy** $\mu(a|s)$

$$\{s_1, a_1, R_2, \dots, s_T\} \sim \mu$$

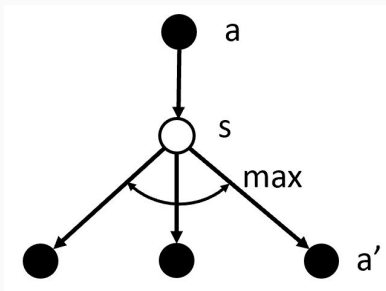
Why is this important?

- Learn from observing humans or other agents
- Re-use experience generated from old policies $\pi_1, \pi_2, \dots, \pi_{t1}$
- Learn about optimal policy while following exploratory policy
- Learn about multiple policies while following one policy

Off-Policy Control with Q-Learning

- We now consider off-policy learning of action-values $Q(s, a)$
- Next action is chosen using the behaviour policy
 $A_{t+1} \sim \mu(|S_t)$ but we update the target policy by considering the possible actions that could be applied to the next state and choose one with maximal value.

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma \max_a Q(S', a) - Q(S, A))$$



Off-Policy Control with Q-Learning

- In Q-learning we allow both behaviour and target policies to improve
- The target policy π is greedy w.r.t. $Q(s, a)$

$$\pi(s) = \arg \max_{a'} Q(s, a')$$

- The behaviour policy μ is (e.g.) ϵ -greedy w.r.t. $Q(s, a)$
- The Q-learning target then simplifies:

$$\begin{aligned} & R + \gamma Q(s', a') \\ &= R + \gamma Q(s', \arg \max_a Q(s', a)) \\ &= R + \max_a \gamma Q(s', a) \end{aligned}$$

Temporal-
Difference
Learning

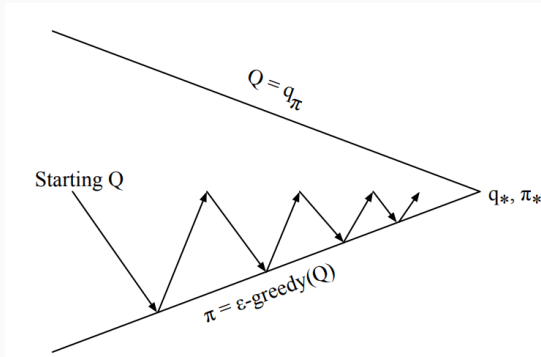
TD Evaluation

TD Control

MC vs. TD

Conclusion

Off-Policy Control With Q-Learning



Every time-step:

- Policy evaluation: Q-learning $Q \approx \epsilon$ -greedy
- Policy improvement: α -greedy improvement

Temporal-Difference Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

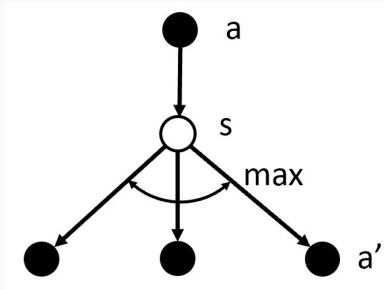
Algorithm 2 Q-Learning (Off-Policy TD Control)

```
1: Initialize:
2:  $Q(s, a) \leftarrow$  arbitrary for all  $s \in \mathcal{S}, a \in \mathcal{A}(s)$ 
3:  $Q(\text{terminal}, \cdot) \leftarrow 0$ 
4: while true do
5:   Initialize start state  $S$ 
6:   while  $S$  is not terminal do
7:     Choose action  $A$  with  $\epsilon$ -greedy policy derived from  $Q$ 
8:     Execute action  $A$ 
9:     Observe reward  $R$  and next state  $S'$ 
10:     $Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma \max_a Q(S', a) - Q(S, A))$ 
11:     $S \leftarrow S'$ 
12:   end while
13: end while
```

$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma \max_a Q(S', a) - Q(S, A))$$

Off-Policy Control with Q-Learning

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma \max_a Q(S', a) - Q(S, A))$$



Theorem

Q-learning control converges to the optimal action-value function, $Q(s, a) \rightarrow Q^*(s, a)$

Temporal-Difference Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

SARSA vs. Q-Learning

Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

SARSA:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A') - Q(S_t, A_t))$$

Q-Learning:

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma \max_a Q(S', a) - Q(S, A))$$

MC vs. TD

- Goal: learn and optimize value function online from experience
- Incremental every-visit Monte-Carlo
 - Update value $V(s_t)$ toward actual return G_t
(typically, $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$)

$$V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$$

- Simplest temporal-difference learning algorithm: $TD(0)$
 - Update value $V(s_t)$ toward estimated return $R_{t+1} + \gamma V(s_{t+1})$

$$V(s_t) \leftarrow V(s_t) + \alpha(R_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$

- $R_{t+1} + \gamma V(s_{t+1})$ is called the **TD target**
- $R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is called the **TD error**

Going Home Example

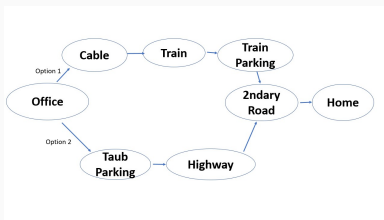
Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion



Going Home Example

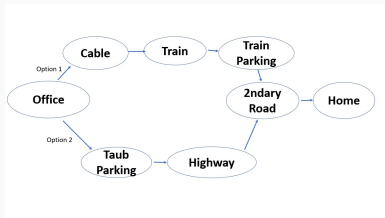
Temporal-
Difference
Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion



- τ_1 = office, 20, cable, 15, train, 30, train-park, 5, 2ndary, 15, home
- τ_2 = office, 20, cable, 15, train, 60, train-park, 5, 2ndary, 15, home
- τ_3 = office, 5, taub-park, 10, highway, 60, 2ndary, 15, home
- τ_4 = office, 5, taub-park, 10, highway, 60, 2ndary, 35, home
- τ_5 = office, 5, taub-park, 10, highway, 60

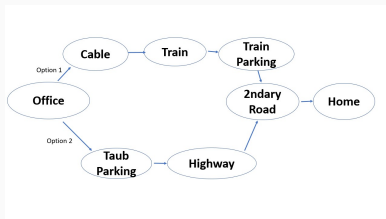
Going Home Example

$$V\pi(s_t) \leftarrow V\pi(s_t) + \alpha(G_t - V\pi(s_t))$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

$$V\pi(s_t) \leftarrow V\pi(s_t) + \alpha(R(s_t) + \gamma V\pi(s_{t+1}) - V\pi(s_t))$$

$$\gamma = 1 \quad \alpha = 0.5$$



- τ_1 = office, 20, cable, 15, train, 30, train-park, 5, 2ndary, 15, home
- τ_2 = office, 20, cable, 15, train, 60, train-park, 5, 2ndary, 15, home
- τ_3 = office, 5, taub-park, 10, highway, 60, 2ndary, 15, home
- τ_4 = office, 5, taub-park, 10, highway, 60, 2ndary, 35, home
- τ_5 = office, 5, taub-park, 10, highway, 60

Temporal-Difference Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

Going Home Example

$$V_{\pi}(s_t) \leftarrow V_{\pi}(s_t) + \alpha(G_t - V(s_t))$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

$$V_{\pi}(s_t) \leftarrow V_{\pi}(s_t) + \alpha(R(s_t) + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t))$$

$$\gamma = 1 \quad \alpha = 0.5$$

		MC		TD	
State	0	1	2	1	2
Exit office					
Enter Cable					
Enter Taub Park					
Enter Train					
Enter Train Park					
Enter Highway					
Enter 2nd Road					
Enter Home					

- τ_1 = office, 20, cable, 15, train, 30, train-park, 5, 2ndary, 15, home
- τ_2 = office, 20, cable, 15, train, 60, train-park, 5, 2ndary, 15, home
- τ_3 = office, 5, taub-park, 10, highway, 60, 2ndary, 15, home
- τ_4 = office, 5, taub-park, 10, highway, 60, 2ndary, 35, home
- τ_5 = office, 5, taub-park, 10, highway, 60

Temporal-Difference Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion

- TD can learn before knowing the final outcome
 - TD can learn online after every step
 - MC must wait until end of episode before return is known
- TD can learn without the final outcome
 - TD can learn from incomplete sequences
 - MC can only learn from complete sequences
 - TD works in continuing (non-terminating) environments
 - MC only works for episodic (terminating) environments

- Return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$ is an **unbiased** estimate of $V\pi(s_t)$
- True TD target $R_{t+1} + \gamma v_\pi(s_{t+1})$ is **unbiased estimate** of $V\pi(s_t)$.
- TD target $R_{t+1} + \gamma V(s_{t+1})$ is a **biased** estimate $V\pi(s_t)$. **why?**
- TD target has much lower variance than the return - **why?**

- Return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$ is an **unbiased** estimate of $V\pi(s_t)$
- True TD target $R_{t+1} + \gamma v_\pi(s_{t+1})$ is **unbiased estimate** of $V\pi(s_t)$.
- TD target $R_{t+1} + \gamma V(s_{t+1})$ is a **biased** estimate $V\pi(s_t)$. **why?**
- TD target has much lower variance than the return - **why?**
 - Return depends on many random actions, transitions, rewards
 - TD target depends on one random action, transition, reward

- MC has high variance, zero bias
 - Good convergence properties
 - (even with function approximation)
 - Not very sensitive to initial value
 - Very simple to understand and use
- TD has low variance, some bias
 - Usually more efficient than MC
 - $TD(0)$ converges to $V\pi(s)$
 - (but not always with function approximation)
 - More sensitive to initial value

- TD exploits Markov property
 - Usually more efficient in Markov environments
- MC does not exploit Markov property
 - Usually more effective in non-Markov environments

- MC converges to solution with minimum mean-squared error
 - Best fit to the observed returns

$$\sum_{k=1}^K \sum_{t=1}^{T_k} (G_t^k - V(s_t^k))^2$$

- $TD(0)$ converges to solution of max likelihood Markov model
 - Solution to the MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ that best fits the data

$$\mathcal{P}_{s,s'}^a = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^{T_k} 1(s_t^k, a_t^k, s_{t+1}^k = s, a, s')$$

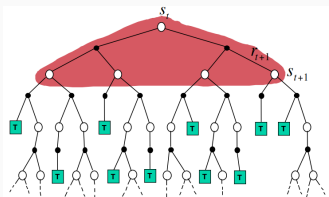
$$\mathcal{R}_s^a = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^{T_k} 1(s_t^k, a_t^k = s, a) r_t^k$$

Conclusion

Monte-Carlo vs. Temporal-Difference vs. Dynamic Programming Backup

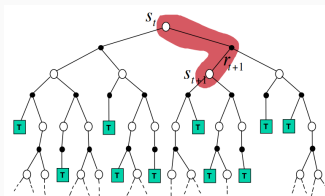
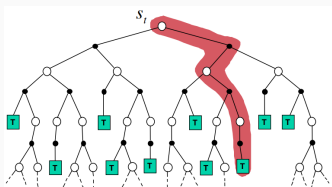
Which is which ?

$$V(s_t) \leftarrow \mathbb{E}_{\pi}[R_{t+1} + \gamma V(s_{t+1})]$$



$$V(s_t) \leftarrow V(s_t) + \alpha(G_t V(s_t))$$

$$V(s_t) \leftarrow V(s_t) + \alpha(R_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$



Learning

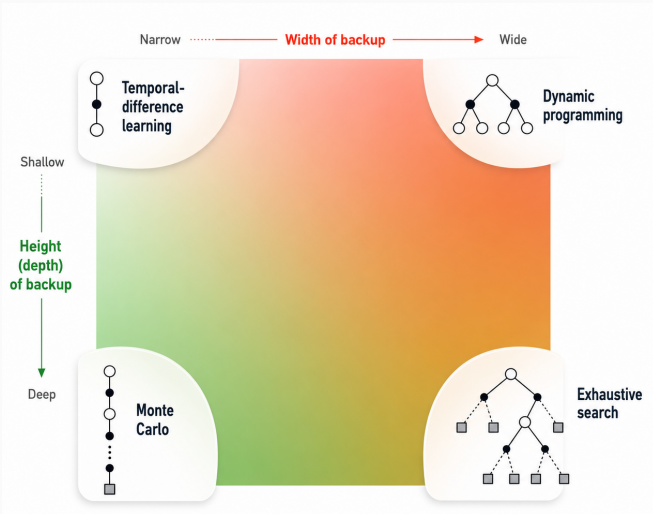
TD Evaluation

TD Control

MC vs. TD

Conclusion

Unified View of Reinforcement Learning



Temporal-Difference Learning

TD Evaluation

TD Control

MC vs. TD

Conclusion